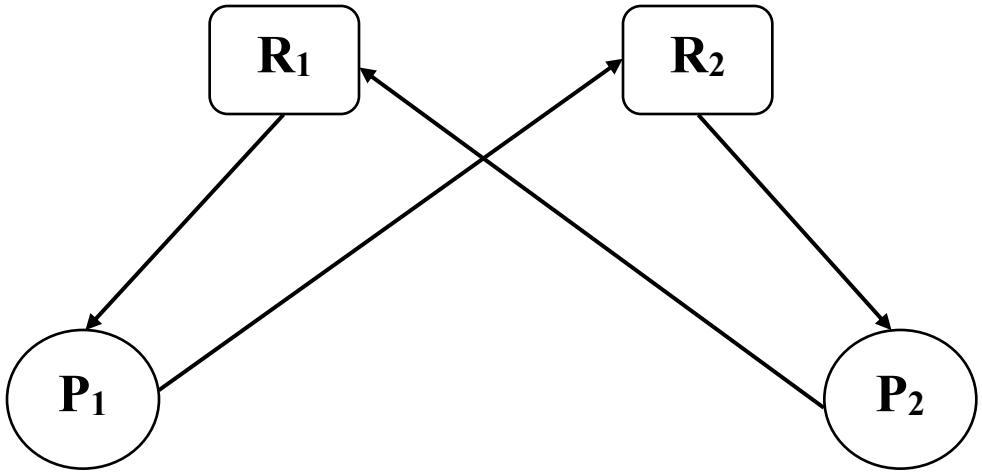
Deadlock occurs in a system when a set of processes are permanently blocked, each waiting for a resource that another process in the set holds. For deadlock to occur, the following four conditions must hold simultaneously:

Conditions for Deadlock:

1. **Mutual Exclusion**:
   - At least one resource must be held in a **non-shareable** mode, meaning only one process can use the resource at a time.
   - If another process requests that resource, it must wait until the resource is released.
2. **Hold and Wait**:
   - A process must be **holding at least one resource** and simultaneously waiting to acquire additional resources that are currently being held by other processes.
3. **No Preemption**:
   - Resources cannot be forcibly taken away from a process. A resource can only be **released voluntarily** by the process holding it, once the process completes its task.
4. **Circular Wait**:
   - There must exist a set of processes {P1, P2, ..., Pn} such that:
     - Process P1 is waiting for a resource held by P2,
     - Process P2 is waiting for a resource held by P3,
     - ...
     - Process Pn is waiting for a resource held by P1.
   - This creates a circular chain or list where each process waits for the next process in the chain to release a resource.



In this situation:

- **Mutual Exclusion:** Both R1 and R2 can only be held by one process at a time.
- **Hold and Wait:** P1 holds R1 and waits for R2, and P2 holds R2 and waits for R1.
- **No Preemption:** Neither P1 nor P2 can be forced to release its resources.
- **Circular Wait**: P1 is waiting for P2, and P2 is waiting for P1—creating a circular dependency.

Methods for Handling Deadlocks:

There are three main approaches to deal with deadlocks:

1. **Ignoring the Problem:**
    - Simply pretend that deadlocks never happen.
    - This is used when deadlocks are rare and the cost of prevention or detection is higher than the impact of the deadlock itself.
    - This is the approach used by most systems (e.g., UNIX).
2. **Deadlock Prevention or Avoidance:**
    - The system uses a protocol to **prevent** or **avoid** deadlocks, ensuring that the system will never enter a deadlock state.

        - **Deadlock Prevention:** Prevent at least one of the four necessary conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) from happening.
        - **Deadlock Avoidance:** Use algorithms (like the Banker's Algorithm) to ensure that the system stays in a safe state where deadlocks cannot occur.
3. **Deadlock Detection and Recovery:**
    - The system **allows deadlocks to occur** but detects them when they happen.
    - Once a deadlock is detected, the system can take action to recover from the deadlock.
        - **Recovery methods** include terminating one or more of the deadlocked processes or preempting resources from some processes to break the deadlock cycle.

Deadlock vs. Starvation

Basis for Comparison	Deadlock	Starvation
Basic Definition	Deadlock is a situation where no process proceeds, and all processes get blocked.	Starvation is when low-priority processes get blocked while high-priority processes proceed.
Arising Condition	Occurs due to mutual exclusion, hold and wait, no preemption, and circular wait simultaneously.	Arises from the enforcement of priorities and uncontrolled resource management.
Other Name	Circular Wait	Livelock
Resource Usage	In deadlock, requested resources are blocked by other processes.	In starvation, requested resources are continuously used by high-priority processes.
Prevention	Avoid mutual exclusion, hold and wait, circular wait, and allow preemption.	Prevention is achieved by aging, where the priority of waiting processes is gradually increased.

Q. List three examples of deadlocks that are not related to a computer- system environment.

1. Traffic Deadlock:

- Imagine a four-way intersection where all cars block each other's path by trying to move forward at the same time. Since none of the cars can proceed without the others moving, they're stuck in a deadlock. This mirrors a situation where resources (the road) are held by different entities (cars) waiting on others to release them.

2. Diners at a Restaurant:

- Suppose two people are dining together, and they each want to pass the salt and pepper. One person grabs the salt while the other grabs the pepper, and each waits for the other to pass their item first. Since both are holding what the other needs and are waiting, this creates a deadlock.

3. Construction Machinery Deadlock:

- Consider two cranes working in close proximity, where one crane holds a load while the other crane needs to move in the same space to continue working. If both operators wait for the other to move their crane first, neither can proceed, causing a deadlock in the construction process.

Banker's algorithm:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Current Need= Max Need - Allocation

Process	Allocation			Max Need			Available			Current Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	3				1	2	3
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

Available need to decide by checking current need, then

Available = Previous Available + Allocation

Process	**Allocation**			**Max Need**			**Current Need**			**Available**		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	3	1	2	3	**5**	**4**	**3**
P2	3	0	2	9	0	2	6	0	0	**7**	**4**	**3**
P3	2	1	1	2	2	2	0	1	1	**7**	**5**	**3**
P4	0	0	2	4	3	3	4	3	1	**10**	**5**	**5**
										10	5	7

<P3-P1-P0-P2-P4>

Banker’s Algorithm Safe.

Request Resource Algorithm:

1. Request ≤ Current need? Goto 2
2. Request ≤ available? Goto 3
3. Available = Available – Request
   Allocation = Allocation + Request
   Current Need = Current Need + Request
4. Check new states is safe or not by using Banker's algorithm

Process	Allocation			Max Need			Available			Current Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	3				1	2	3
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

#) If P1 requests (1,0,2), determine if it can be granted immediately.

1) Req <1,0,2> ≤ Current Need <1,2,3>
2) Req <1,0,2> ≤ Available <3,3,2>
3) Available<2,3,0> = Available<3,3,2> – Request<1,0,2>
   Allocation<3,0,2> = Allocation<2,0,0> + Request<1,0,2>
   Current Need<2,2,5> = Current Need<1,2,3> + Request<1,0,2>

4) Banker's algo:

Process	**Allocation**			**Max Need**			**Current Need**			**Available**		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	2	3	0
P1	3	0	2	3	2	3	2	2	5	4	**4**	**1**
P2	3	0	2	9	0	2	6	0	0	**4**	**4**	**3**
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

<P3, P4>

The process is unsafe. Deadlock may occur.